

Module 1 — Applied Lab: lab-1-git-workflows

Lab type: Applied Lab (autograded via GitHub Actions) **Submission:** GitHub PR URL → TalentLMS Assignment (Unit 5) **Points:** 35

Overview

A three-person team has been asked to analyze hospital admission records for a regional health authority. Before writing a single line of analysis code, the team needs a shared, reproducible project environment — one that any team member or TA can spin up in under two minutes using two commands.

Your job is to configure that environment: create a `requirements.txt`, set up a local venv, confirm the environment works, and push everything through a proper Git feature-branch workflow.

By completing this lab, you will:

- Create a `requirements.txt` that specifies the project's Python dependencies
- Configure and verify a local venv using pip
- Use a Git feature-branch workflow: branch → commit → push → pull request
- Pass a set of automated checks that verify your repository is correctly structured

Setup

Step 1: Accept the Assignment

[Click here](#) to create your personal copy of the `aspire-foundations` starter repo on GitHub with the Module 1 lab files already in place.

GitHub will give you a repo URL in the form:

```
https://github.com/LevelUp-Applied-AI/aspire-foundations-<your-username>
```

Step 2: Clone Your Repo

```
git clone <your-repo-url>
cd aspire-foundations-<your-username>
```

Step 3: Inspect the Starter Files

The repo contains four starter files:

File	What it is
<code>README.md</code>	Stub with <code>TODO:</code> markers — you'll update this
<code>requirements.txt</code>	Empty stub with a comment — you fill this in
<code>.gitignore</code>	Minimal Python starter — you must expand it
<code>test_environment.py</code>	Provided and complete — do not modify

Read `test_environment.py` before continuing. It imports `pandas` and `matplotlib` and prints `Environment OK` on success. It exits with a non-zero code if either import fails. The autograder runs this script — if it doesn't pass, the lab doesn't pass.

Step 4: Create Your Branch

Before making any changes, create your feature branch:

```
git checkout -b setup/environment
```

You should now see `setup/environment` as your active branch (`git branch` to verify).

Your Task

Context

The hospital admission data project needs a shared environment that any team member can recreate exactly. This means: a `requirements.txt` specifying the project's dependencies, a `.gitignore` that keeps the venv and data files out of Git, and confirmation that the environment actually works before anyone else has to set it up.

What to Build

1. Fill in `requirements.txt`

Add the two packages the project needs (one per line, lowercase):

```
pandas
matplotlib
```

Version pins are optional at this stage. The autograder checks for package names, not pinned versions.

2. Expand `.gitignore`

The starter `.gitignore` has `__pycache__/, *.pyc,` and `.ipynb_checkpoints/`. Add `.venv/` at minimum. See the reading (Section 8.3) for the full list of recommended patterns.

3. Create and activate your venv

```
python -m venv .venv
```

Then activate it using the command for your OS:

OS	Shell	Command
Mac / Linux	bash or zsh	<code>source .venv/bin/activate</code>
Windows	Git Bash	<code>source .venv/Scripts/activate</code>
Windows	CMD	<code>.venv\Scripts\activate.bat</code>
Windows	PowerShell	<code>.venv\Scripts\Activate.ps1</code>

Confirm `(.venv)` appears in your terminal prompt before continuing.

4. Install the requirements and verify

```
pip install -r requirements.txt
python test_environment.py
```

You should see:

```
Environment OK
```

If you see a `ModuleNotFoundError`, the venv is not active or a package is missing from `requirements.txt`. Do not proceed until this passes.

5. Update `README.md`

Replace the heading and at least one `TODO:` marker with real content. The README does not need to be complete for the lab — full README completion is the integration task. At minimum:

- Replace [Project Title — e.g., Hospital Admission Records Analysis] in the heading with the actual project name
- Add a brief description (1–2 sentences about the hospital admission analysis project) in the Project Overview section

Team member names are not required for the lab — they go in Section 2 of the completed README in the integration task.

Step-by-Step Guide

Step 1 — Write requirements.txt

Open `requirements.txt` in your editor. It currently contains a comment. Replace the file contents with:

```
pandas
matplotlib
```

Save the file.

Step 2 — Update .gitignore

Open `.gitignore` and add `.venv/` on its own line. The autograder checks that `.venv/` or `.venv` appears in this file. **Do not commit the `.venv/` directory — it must be excluded from Git.**

If you want to confirm `.venv/` is being ignored before committing:

```
# After creating .venv/, run:
git status
```

The `.venv/` directory should NOT appear in the untracked files list. If it does, `.gitignore` is not picking it up — check that `.venv/` is on its own line with no extra spaces.

Step 3 — Create venv, Install, Verify

```
python -m venv .venv
source .venv/bin/activate # Mac/Linux
pip install -r requirements.txt
python test_environment.py
```

Expected output: `Environment OK`

Step 4 — Commit Your Changes

Stage only the files you changed (never `git add .` or `git add -A` until you've reviewed `git status`):

```
git status
git add requirements.txt .gitignore README.md
git commit -m "Add requirements.txt and configure project environment"
```

Write a real commit message. "init" or "done" will be visible to the autograder and your TA.

Step 5 — Push and Open a Pull Request

```
git push --set-upstream origin setup/environment
```

Then go to your repo on GitHub. You'll see a "Compare & pull request" banner. Click it:

- **Base branch:** `main`
- **Compare branch:** `setup/environment`
- **Title:** `Lab 1 — Environment Setup`
- **Description:** Include what the environment contains, the commands needed to activate the venv and run `test_environment.py`, and confirmation that `test_environment.py` passes locally. Example:

```
Added requirements.txt (pandas, matplotlib) and expanded .gitignore to exclude
Updated README with project title and description.

To verify: activate .venv, run 'pip install -r requirements.txt', then 'python
```

Click "Create pull request." The autograder will run within a few seconds.

Autograder

The autograder runs automatically when you push to `setup/environment` and when you open the PR. It checks:

Check	What it verifies
Required files present	<code>requirements.txt</code> , <code>README.md</code> , <code>.gitignore</code> , <code>test_environment.py</code> all exist in the repo
<code>requirements.txt</code> contains required packages	<code>pandas</code> and <code>matplotlib</code> appear in the file (case-insensitive)
<code>.gitignore</code> excludes <code>.venv/</code>	<code>.venv</code> or <code>.venv/</code> appears in <code>.gitignore</code>
<code>test_environment.py</code> passes	Script runs with exit code 0 and stdout contains <code>Environment OK</code>
<code>README.md</code> is non-empty	File is more than 50 bytes (i.e., not just a blank title line)

If a check fails: Click on the red *X* next to your commit (or on the "Checks" tab in the PR) to see the GitHub Actions log. The error message will identify exactly which check failed and why. Fix the issue, commit, and push — the autograder reruns automatically.

Do not submit the PR URL to TalentLMS until all checks are green ☑.

Submitting

1. Confirm all autograder checks pass (green ☑ on the PR)
2. Copy the PR URL from your browser address bar
3. Submit to **TalentLMS → Module 1 → Lab 1 (Unit 5)**

The PR must be open (not merged) at the time of submission. Do not merge your own PR.

Troubleshooting

If `test_environment.py` fails with `ModuleNotFoundError`: Confirm `(.venv)` is in your prompt. If not, re-activate. Then check that `requirements.txt` contains `pandas` and `matplotlib` (no typos), and re-run `pip install -r requirements.txt`.

If `.venv/` shows up in `git status` as untracked: You forgot to add `.venv/` to `.gitignore`, or the entry has a typo. Open `.gitignore` and add `.venv/` on its own line. Run `git status` again — it should disappear from the untracked list.

If `git push` is rejected with "non-fast-forward": Someone (or an initial commit) created commits on `main` after you branched. Run `git pull origin main`, resolve any conflicts if needed, then push again.

If the autograder shows a **YAML syntax error**: You probably have a tab character in `.gitignore` or `requirements.txt`. Open the file in your editor and check for invisible characters. Both files should use plain text with Unix line endings.

If the autograder can't find a file: Confirm the file exists in the repo root (not in a subdirectory). Run `ls` (Mac/Linux) or `dir` (Windows) in your repo folder and verify the file name exactly — including capitalization. `Requirements.txt` will not match `requirements.txt`.

Post in the course channel if you're stuck after trying these steps. Include your PR URL and a description of the failing check.

